
Guia de Instalação — MobU (Backend e Plataforma Web)

Este guia descreve, passo a passo, como colocar o **backend (API)** e o **painel administrativo web** do MobU para funcionar em um computador. Ele assume pouca familiaridade com programação: cada etapa explica **o que** está sendo feito e **por quê**. Para instruções resumidas, consulte o **README** na raiz do repositório.

1. Entendendo as peças do sistema

O MobU é formado por 4 partes que conversam entre si:

Peça	O que é	Onde roda
Banco de dados (PostgreSQL)	Onde ficam guardados usuários, corridas, pagamentos etc.	No computador, dentro do Docker
Backend / API (NestJS)	O "cérebro": recebe os pedidos do app e do painel, aplica as regras e grava no banco	No computador, porta 3000
Painel administrativo (Next.js)	O site usado pelo administrador	No computador, porta 3001
Aplicativo Android	O app do passageiro e do motorista	No celular ou emulador

A ordem de instalação é exatamente essa: primeiro o banco, depois o backend, depois o painel e por fim o aplicativo.

2. Programas necessários (pré-requisitos)

Instale antes de começar:

1. **Node.js versão 20 ou superior** — é o ambiente que executa o backend e o painel. Para conferir se já tem, abra o terminal e digite `node -v`.
2. **Docker Desktop** — programa que roda o banco de dados em um "contêiner", sem precisar instalar o PostgreSQL manualmente. Após instalar, **deixe o Docker aberto**.

3. **Android Studio** — apenas se você for rodar o aplicativo Android a partir do código-fonte.

Você também precisará de duas contas gratuitas (com chaves de acesso):

1. **Google Cloud** com a **Maps API habilitada** — o sistema usa o Google Maps para calcular rotas e preços. Crie uma chave de API no console do Google Cloud (APIs necessárias: Directions/Routes e Maps).
2. **Firebase** com **Cloud Messaging habilitado** — serviço do Google usado para enviar notificações push ao aplicativo.

3. Passo 1 — Banco de dados (PostgreSQL via Docker)

Abra o **terminal** na pasta do projeto e execute:

```
cdCodigo
docker compose up -d
```

O que isso faz: baixa e inicia o PostgreSQL 16 em segundo plano (`-d` = *detached*). O banco ficará disponível em `localhost:5432`.

Como saber se deu certo: execute `docker ps` — deve aparecer um contêiner com "postgres" na lista.

4. Passo 2 — Backend (API NestJS)

4.1 Configurar as variáveis de ambiente

O backend lê suas configurações de um arquivo chamado `.env`. Crie-o a partir do modelo:

```
cdCodigo/backend/api
cp .env.example .env
```

Abra o `.env` em qualquer editor de texto e preencha:

Variável	O que colocar
DATABASE_URL	Endereço do banco (o exemplo do <code>.env.example</code> já aponta para o Docker local — normalmente basta manter)
JWT_SECRET	Qualquer texto longo e aleatório (é o "segredo" que protege os logins; nunca compartilhe)
GOOGLE_MAPS_API_KEY	A chave criada no Google Cloud (seção 2, item 4)
Credenciais do Firebase	Dados da conta de serviço do Firebase, para as notificações push

4.2 Instalar e preparar

Ainda na pasta `Codigo/backend/api`:

```
npm install           # baixa as dependências do projeto (pode
                      # demorar alguns minutos)
npx prisma migrate deploy # cria as tabelas no banco de dados
npx prisma db seed      # cria o usuário administrador e a tarifa
                      # inicial
```

O que o seed cria: um administrador padrão (telefone `31999000000`, senha `Senha@123`) e a configuração inicial de tarifas — sem isso não é possível entrar no painel nem estimar corridas.

4.3 Iniciar

```
npm run start:dev
```

A API estará no ar em `http://localhost:3000`. Deixe esse terminal aberto — fechá-lo derruba o backend.

Como saber se deu certo: o terminal mostrará mensagens do NestJS sem erros, terminando com algo como "Nest application successfully started".

4.4 Testes (opcional)

```
npm test           # testes unitários
npm run test:e2e    # testes de ponta a ponta
```

5. Passo 3 — Painel administrativo (Next.js)

Em **outro terminal** (deixe o backend rodando no primeiro):

```
cd Codigo/dashboard/admin
cp .env.local.example .env.local # configure a URL da API (http://
    localhost:3000) e a chave do Maps
npm install
npm run dev
```

Abra o navegador em **http://localhost:3001** e entre com o administrador padrão: telefone **31999000000** / senha **Senha@123**.

⚠ Em produção, troque essa senha imediatamente.

6. Passo 4 — Aplicativo Android

1. Abra o **Android Studio** e, nele, abra a pasta **Codigo/MobULite**.
2. Copie o arquivo **local.properties.example** para **local.properties** e preencha a chave do Google Maps.
3. Configure o endereço da API no arquivo **RetrofitClient.kt**:
 - **Emulador** no mesmo computador: use **http://10.0.2.2:3000** (endereço especial que o emulador usa para "enxergar" o computador);
 - **Celular físico**: use o IP do computador na rede local (ex.: **http://192.168.0.10:3000**) — celular e computador precisam estar no **mesmo Wi-Fi**.
4. Clique em **Run** (▶) e escolha um emulador ou o celular conectado por USB (com *depuração USB* ativada).

O aplicativo requer **Android 8.0 (API 26)** ou superior.

7. Ordem de inicialização no dia a dia

Sempre que for usar o sistema, inicie nesta ordem:

1. **Docker** (banco): **docker compose up -d** na pasta **Codigo** (se ainda não estiver rodando);

2. **Backend:** `npm run start:dev` em `Codigo/backend/api`;
 3. **Painel:** `npm run dev` em `Codigo/dashboard/admin`;
 4. **App:** Run no Android Studio.
-

8. Solução de problemas

Problema	Causa provável e o que verificar
<code>docker compose</code> falha	O Docker Desktop está aberto? Reinicie-o e tente de novo.
API não conecta ao banco (<code>Can't reach database</code>)	O contêiner do banco está rodando (<code>docker ps</code>)? A <code>DATABASE_URL</code> do <code>.env</code> está correta?
Erro <code>GOOGLE_MAPS_API_KEY</code> não configurada	Preencha a chave no <code>.env</code> e reinicie o backend.
Estimativa de corrida falha	A chave do Maps é válida? As APIs de rotas estão habilitadas no Google Cloud? Há região de operação cadastrada cobrindo o local?
Push não chega no app	Credenciais do Firebase no backend e arquivo <code>google-services.json</code> no app estão configurados?
App não acessa a API	O endereço em <code>RetrofitClient.kt</code> está certo para o seu caso (emulador × celular físico)? Firewall do computador liberou a porta 3000?
Painel abre mas o login falha	O seed foi executado (<code>npx prisma db seed</code>)? O backend está rodando?
Porta 3000/3001 ocupada	Outro programa usa a porta — feche-o ou reinicie o computador.